Software Engineering Project · AI-Powered Co-Working Space Asset Manager

Hooria's Task Guide

AI screens + Notifications + Usage History · partner: Misbah

Your job in one sentence

You build the **screens** for the two AI features (Misbah's server part is already finished and working), plus the **Notifications** and **Usage History** modules from start to finish (server and screens).

Your files (only touch these)

File	What you put in it
`client/src/modules/ai/RecommendPage.jsx`	AI Recommendation screen
`client/src/modules/ai/AssistantPage.jsx`	AI Assistant chat screen
`server/src/modules/notifications/routes.js`	4 notification endpoints
`server/src/modules/notifications/jobs.js`	"Due tomorrow" reminder
`server/src/services/notify.js`	Already works. Keep the names `notify` and `notifyAdmins`
`client/src/modules/notifications/NotificationBell.jsx`	Bell in the top bar with the unread count
`client/src/modules/notifications/NotificationsPage.jsx`	List of notifications
`server/src/modules/history/routes.js`	2 history endpoints
`client/src/modules/history/MyHistoryPage.jsx`	Member's own history
`client/src/modules/history/AssetHistoryPage.jsx`	Admin: history of one asset

You may add new files inside these folders. Do not edit other people's folders, `app.js`, `routes.jsx` or the database schema. If you need a change there, ask Shahan.

The order to work in

Step 0
Run the app on your laptop

Step 1
Notifications (server, then bell and page)

Step 2
Usage History (server, then 2 pages)

Step 3
AI Recommendation screen

Step 4
AI Assistant chat screen

Why this order: Notifications and History are small and teach you the pattern (endpoint, then screen). The AI screens come last because they are the biggest, and Misbah's server part is ready whenever you are.

Three rules: (1) Keep every path and response shape exactly as written in `docs/API.md`, because other people's code uses them. (2) Build every screen from the UI template (page 4). (3) Check your screen in light and dark mode.

0 Run the app

You need **Node.js 20+** and **MySQL 8** (or XAMPP's MySQL). Open two terminals:

```
# Terminal 1: server
cd server
npm install
copy .env.example .env      # Mac/Linux: cp .env.example .env
                            # open .env, set DB_USER and DB_PASSWORD to your MySQL login
npm run db:setup            # creates the database with sample data
npm run dev                 # server on http://localhost:5000

# Terminal 2: website
cd client
npm install
npm run dev                 # open http://localhost:5173
```

Log in: admin `admin@cowork.local` / `Admin@123` · member `member@cowork.local` / `Member@123`

Your pages show as placeholder cards with your name; your endpoints answer "501" until you write them. **AI key:** without `ANTHROPIC_API_KEY` in `.env`, Recommendation still works (simple keyword ranking) and the chat returns an "unavailable" error, which is useful for testing your error message.

1 Notifications `FR-NT-01 to 03`

a) Server: `notifications/routes.js`

Replace each `todo` with a real handler. Every query uses only the logged-in user's rows (`req.user`).

Endpoint	Returns
`GET /api/notifications`	`[{ notification_id, type, message, is_read, created_at }]`, newest first
`GET /api/notifications/unread-count`	`{ count }`
`PATCH /api/notifications/:id/read`	`{ message }`
`PATCH /api/notifications/read-all`	`{ message }`

```
const db = require('../../config/db');   // add at the top

// example: the list
router.get('/', async (req, res) => {
  const [rows] = await db.query(
    'SELECT notification_id, type, message, is_read, created_at FROM notifications ' +
    'WHERE user_id = ? ORDER BY created_at DESC', [req.user.user_id]);
  res.json(rows);
});
```

b) Server: `notifications/jobs.js`

Fill in `sendDueReminders()`: find active borrowings due **tomorrow** and call `notify(...)` once for each borrower (skip anyone who already got a `due_reminder` today).

c) Screens

- [ ] **Bell** (`NotificationBell.jsx`): call `/notifications/unread-count` every 60 seconds and show the number in a small dot. Hide the dot at 0.
- [ ] **Page** (`NotificationsPage.jsx`): the list, unread ones in bold or highlighted, a "Mark as read" on each and a "Mark all as read" button.

2 Usage History `FR-UH-01 to 03`

Endpoint	Who	Returns
`GET /api/history/mine`	anyone	`{ requests, borrowings }`: my requests (asset name, dates, status) and my borrowings (issued, due, returned, condition)
`GET /api/history/asset/:id`	admin	`{ asset, borrowings, maintenance }`: every borrowing of that asset with the borrower's name, and every repair

- [ ] **My History** (`MyHistoryPage.jsx`): two tabs or two tables: "My requests" and "My borrowings".
- [ ] **Asset History** (`AssetHistoryPage.jsx`): opened at `/admin/history/<id>` from Azlaan's asset details. Asset name at the top, then borrowings and maintenance.
- [ ] **Read only:** no edit or delete buttons anywhere (FR-UH-03).

3 AI Recommendation screen `FR-RC-01, 04, 05, 06`

The member describes what they need, and the server returns the best free assets.

```
const res = await api.post('/ai/recommend', {
  requirement: 'laptop for video editing', purpose: 'Client video',
  start_date: '2026-11-02', duration_days: 3 });
// res = { source: 'ai' or 'rules', start_date, duration_days, recommendations: [
//   { rank, reason, asset: { asset_id, name, category, location, condition, image_path } } ],
//   message?, nearest_date? }
```

- [ ] Form: requirement (box, max 500 characters), purpose, start date (today or later), number of days (1 to 30).
- [ ] While waiting, show a spinner and "Finding the best match..." (it can take up to 10 seconds).
- [ ] Show up to 5 result cards in order: picture, name, category, location, condition and the AI's **reason**.
- [ ] If `source` is `'rules'`, show a small note: "Ranked by keyword match".
- [ ] Each card has a **Request** button that links to `/request/<asset_id>?start=<start_date>&days=<duration_days>`. Admins don't see it.
- [ ] No results: show `message`, and if `nearest_date` is set, offer "Search from <nearest_date>".

4 AI Assistant chat screen `FR-AI-01, 07`

```
const res = await api.post('/ai/chat', { question, history });
// history = earlier messages, oldest first: [{ role: 'user' or 'assistant', content: 'text' }]
// res = { answer, links: [{ label: 'View Dell XPS 15', href: '/assets/1' }] }
const usage = await api.get('/ai/usage');   // { used, limit, remaining, ai_available }
```

- [ ] Chat bubbles: the user on the right, the assistant on the left. Text box with a "123 / 500" counter.
- [ ] Show "Thinking..." while waiting. Show each answer's `links` as small buttons under it.
- [ ] Errors (503 AI unavailable, 429 daily limit) appear as a message **inside the chat**, using the server's `error` text. Never a blank page.
- [ ] "12 of 50 AI requests used today" from `/ai/usage`, refreshed after each answer.
- [ ] Example question buttons: "Is any projector free tomorrow?", "Which laptops are on floor 2?", "What are my requests?"

How to build any screen (the UI template)

1. Log in, open the user menu (top right) and choose **UI template** to see every piece and colour.
2. Copy `client/src/modules/_template/PageTemplate.jsx` into your folder and use it as a starting point.
3. In your page file (e.g. `NotificationsPage.jsx`), replace the `<ComingSoon ... />` placeholder with your screen. Keep the file name and the `export default`.
4. Swap the sample rows for your real call, for example `setRows(await api.get('/notifications'))`.
5. Import everything from one place: `import { Page, Card, Button, Table, ErrorBox, EmptyState } from '../../components/ui'`.
6. Use the theme colours (`text-primary`, `bg-muted`), never hex codes. Test in light and dark mode and on a phone width.

Full guide: `docs/UI_TEMPLATE.md`. Pop-up messages: `toast.success('Marked as read')` from `sonner`. Icons: `lucide-react` (Bell, Sparkles, Wand2, History...).

Testing an endpoint without a screen: log in with Postman (`POST /api/auth/login`), copy the `token`, and send it as the header `Authorization: Bearer <token>`.

How you know you're done

- [ ] The bell shows a number, and clicking it opens your Notifications page. "Mark all as read" makes the number go to 0.
- [ ] When an admin approves a member's request, that member's bell count goes up within a minute.
- [ ] A member sees their own requests and borrowings on My History. An admin can open the history of any asset.
- [ ] A member who types `/admin/history/1` in the address bar is sent back to the home page.
- [ ] "Laptop for video editing, 3 days" gives ranked results with reasons, and **Request** opens Hamza's form with the dates filled in.
- [ ] Stopping the server in the middle of a chat shows an error message in the chat, not a blank page.
- [ ] No placeholder cards with your name are left, and every screen looks right in light and dark mode.

Then: push your work to your own branch on GitHub and tell Shahan, who combines everyone's modules.

Stuck? Look here

Question	Where to look
Exact request and response shapes	`docs/API.md` ("Notes for the screens" is written for you)
Your full task list with SRS IDs	`docs/modules/hooria.md`
How the AI server works	`docs/modules/misbah.md`, or ask Misbah
What a requirement means	`docs/SRS.pdf` (search the ID, e.g. FR-NT-01)